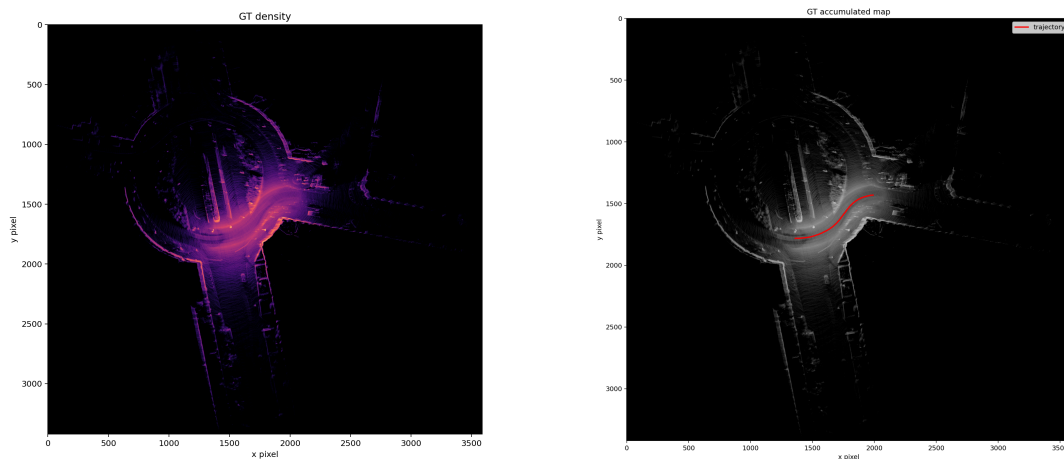


CS 588 MP1 Report: Static Mapping, ICP, and Graph-SLAM
Name: Adwaith Moothethath
NetID: adwaith3

Task 0: Map Alignment and Rasterization

For this task 0 I implemented a point cloud accumulation to world pose and rasterize it to a top down image. So it transforms local lidar scans into a global coordinate frame using SE(2) ground truth poses and projects them into a 2D Bev density map. This is the output I got.



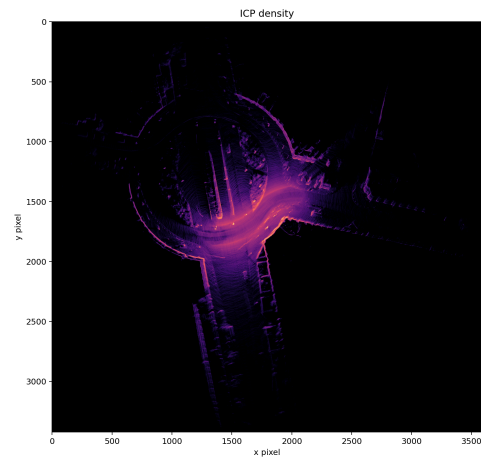
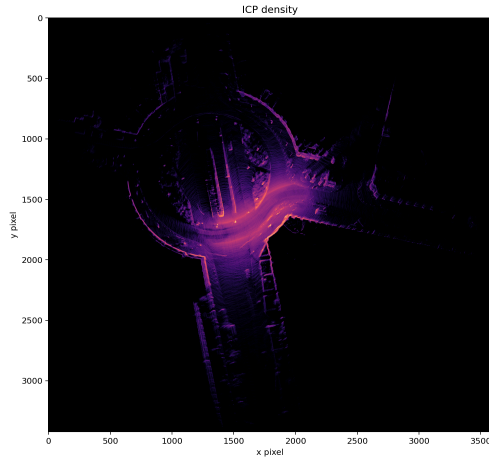
Task 1: ICP Pose Chaining

For this task 1, built the car trajectory by using ICP to measure the relative step between each lidar scan. Also chaining these steps together to track the cars global position and angle.

Questions: Observe the ICP map vs the GT map carefully. What are some noticeable artifacts that you observe in the ICP map and trajectory?

Ans:

- One noticeable artifact is cumulative error. The GT map shows a smooth trajectory but not in the ICP map. This is because since it is chaining one frame to next when there is an error in one frame the next frame builds on that error and it gets worse.
- Also the ICP path trajectory starts to drift away from the center of the road.
- Also there is a lot of ghosting in the walls of the ICP. the walls and edges look blurry compared to Gt map. This is because if one frame is tilted too much after every single frame is tiled.



Task 2.1: Motion Residual and its Jacobian

For task 2.1 implemented an error check to see where the car is to where it is predicted to be. Next I derived Jacobian matrices to tell the optimizer how to adjust the car's position and angle to minimize that error.

Questions: Please write down the analytical solution for J_i and J_j in (you can use LaTeX or handwritten, and include a screenshot/photo). What is the shape of J_i and J_j ?

Ans:

$$J_i = J_t = \frac{\partial r_{\text{motion}}}{\partial x_t}, \quad J_j = J_{t+1} = \frac{\partial r_{\text{motion}}}{\partial x_{t+1}}$$

We know that r_{motion} is the difference between where the car is now and where it is expected it would be

$$r_{\text{motion}} = \begin{bmatrix} r_x \\ r_y \\ r_\theta \end{bmatrix} = \begin{bmatrix} x_{t+1} - (x_t + \cos \theta_t \Delta x_t - \sin \theta_t \Delta y_t) \\ y_{t+1} - (y_t + \sin \theta_t \Delta x_t + \cos \theta_t \Delta y_t) \\ \text{normalize}(\theta_{t+1} - (\theta_t + \Delta \theta_t)) \end{bmatrix}$$

Since J_j is derivative of the residual with respect to $x_{t+1}, y_{t+1}, \theta_{t+1}$ $\frac{\partial r_x}{\partial x_{t+1}} = 1-0, \frac{\partial r_y}{\partial y_{t+1}} = 1-0, \frac{\partial r_\theta}{\partial \theta_{t+1}} = 1-0$ So $J_j = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Since J_i is derivative of the residual with respect to x_t, y_t, θ_t $\frac{\partial r_x}{\partial x_t} = -1, \frac{\partial r_x}{\partial \theta_t} = \frac{\partial [-\cos \theta_t \Delta x_t + \sin \theta_t \Delta y_t]}{\partial \theta_t} = \sin \theta_t \Delta x_t + \cos \theta_t \Delta y_t$

$$\text{Row 2: } \frac{\partial r_y}{\partial y_t} = -1, \quad \frac{\partial r_y}{\partial \theta_t} = \frac{\partial [-\sin \theta_t \Delta x_t - \cos \theta_t \Delta y_t]}{\partial \theta_t} = -\cos \theta_t \Delta x_t + \sin \theta_t \Delta y_t$$

$$\text{Row 3: } \frac{\partial r_\theta}{\partial \theta_t} = -1$$

$$\text{So } J_i = \begin{bmatrix} -1 & 0 & \sin \theta_t \Delta x_t + \cos \theta_t \Delta y_t \\ 0 & -1 & -\cos \theta_t \Delta x_t + \sin \theta_t \Delta y_t \\ 0 & 0 & -1 \end{bmatrix}$$

The shape of Jacobian matrix is: # of residuals x # of state variables
(r_x, r_y, r_θ) (x, y, θ)

So shape of both J_i and $J_j = 3 \times 3$

Task 2.2: Gauss Newton Solver

For task 2.2 implemented Gauss-Newton loop by making the damped linear system from the Jacobians and residuals and then iteratively adjusting the car to minimize the total energy.

Question: Observe the Jacobian output `outputs\jacobian\jacobian spy.png` . Please explain what each line represent. (Hint: Each residual contributes a block row to the Jacobian.)

Ans: This is a sparsity plot showing how different parts of the car's world are connected. A blue dot represents a spot where a residual connects two things, most of the map is empty, which is why it is sparse.

- The main diagonal line in the top left shows the movement over time. The rows show the motion residuals between pose t and $t+1$, and the columns show the corresponding pose variables (x , y , θ). This is the chain that holds the trajectory together.
- The scattered lines show the landmark measurements. The rows are the distance residuals, and the columns show how each line connects a pose column to a landmark column. So, a single line means that at this specific pose, the car saw that specific landmark. It can only see a few landmarks at a time, not everything all together.
- The diagonal line towards the bottom right shows the landmarks positions in the state vector. It is a diagonal line because each landmark is an independent point, so when observed, it is like seeing specific coordinates in the map.

Optimal Bonus Question [+1pt]: You might notice that the linear solver `np.linalg.solve` could be slow. For bonus question, use Cholesky and/or QR Decomposition to speed up. Please profile and report the speed-up performance.

Ans:

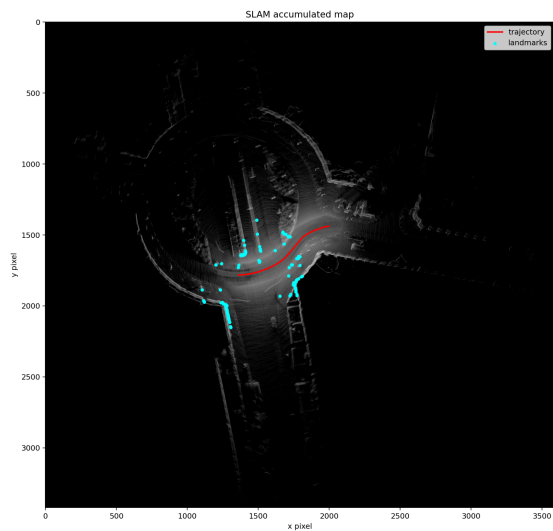
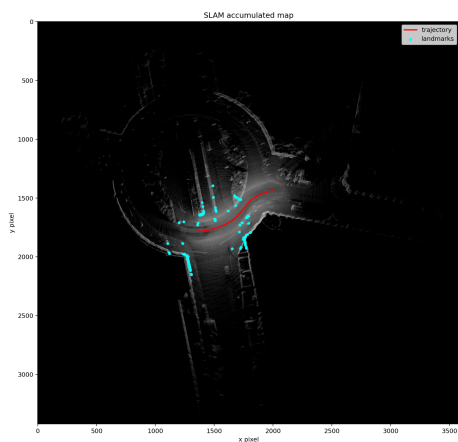
- Using `np.linalg.solve` the total **optimization time was 400.56 ms** and **average iteration time was 66.76 ms**
- Using **Cholesky** the **total optimization time was 76.83 ms** and **average iteration time was 12.80 ms**

Np.linalg.solve results

```
(cs588) PS C:\Users\adwai\Documents\Adwaith\UIUC\av\CS588-SP26\mp1> python .\run.py all
data_root=data/kitti_raw
sequence=2011_09_26_drive_0005_sync
frames=[0, 100) count=100
params: voxel_size=0.25 icp_max_corr=1.5 icp_iters=60sigma_xy=0.24 sigma_theta=0.04 sigma_d=0.08 gn_iters=50
Elapsed time for iteration: 105.87390000000596 ms
Elapsed time for iteration: 34.36759999021888 ms
Elapsed time for iteration: 35.02459998708218 ms
Elapsed time for iteration: 34.557800041511655 ms
Elapsed time for iteration: 147.3699999623932 ms
Elapsed time for iteration: 43.367300007957965 ms
(cs588) PS C:\Users\adwai\Documents\Adwaith\UIUC\av\CS588-SP26\mp1>
```

Cholesky results

```
(cs588) PS C:\Users\adwai\Documents\Adwaith\UIUC\av\CS588-SP26\mp1> python .\run.py all
data_root=data/kitti_raw
sequence=2011_09_26_drive_0005_sync
frames=[0, 100) count=100
params: voxel_size=0.25 icp_max_corr=1.5 icp_iters=60sigma_xy=0.24 sigma_theta=0.04 sigma_d=0.08 gn_iters=50
Elapsed time for iteration: 33.85920001892373 ms
Elapsed time for iteration: 8.226900012232363 ms
Elapsed time for iteration: 8.28359997831285 ms
Elapsed time for iteration: 8.581200032494962 ms
Elapsed time for iteration: 9.287199995014817 ms
Elapsed time for iteration: 8.5873999632895 ms
```



Extra credit: Meshifying and LiDAR Simulation

I implemented a 3D reconstruction by using Poisson reconstruction to turn my slam landmarks into a continuous mesh with a solid surface. To test the map, I then ran a simulation of a Velodyne HDL-64E sensor moving through the map using Open3D raycasting along the ground truth trajectory

3d reconstitution

